CoreRelm

.NET 8+ | MySQL 8+ | docs API + Quickstart | status active development

CoreRelm is a lightweight, attribute-based ORM for C# that stays close to ADO.NET style development while reducing repetitive data access code.

It is designed for developers who want:

- Strong control over SQL and transactions
- Attribute-driven model mapping
- A simple context and dataset workflow
- Predictable behavior without heavy abstraction
- A clear path from model definition to database operations

CoreRelm emphasizes explicitness over magic. You can use high-level helpers when they help, but you are still close to the database and in control of what happens.

API Documentation: https://jdaugherty-bdl.github.io/CoreRelm/index.html

Table of Contents

- Why CoreRelm
- Supported and Current Status
- Quick Start in 60 Seconds
- Features
- What the Quickstart demonstrates
- Feature Matrix
- What is documented here
- Quickstart examples
- Migrations
- Error handling
- CoreRelm vs EF and Dapper
- Contributing
- Roadmap
- Related project

Why CoreRelm

CoreRelm is intended for projects that want a practical middle ground:

- More structure and mapping support than writing raw ADO.NET everywhere
- More explicit control and transparency than heavier ORMs
- A consistent attribute-based model system for schema and query usage

- A simple, readable API for common CRUD and data-loading patterns

If you prefer seeing what the call looks like and understanding what happens, CoreRelm is built for that style.

Supported and Current Status

> **Support**
>
> - CoreRelm supports **.NET 8+**
> - Database provider: **MySQL 8+ only**
>
> **Status**
>
> - Active development
> - API surface is evolving
> - Quickstart examples are documentation-style examples and may not be intended to compile or run as-is
>
> **Packaging**
>
> - Project/reference usage is supported
> - Additional packaging/distribution options may be added over time

Quick Start in 60 Seconds

This section shows the shape of a typical CoreRelm setup.

> These examples are intended to demonstrate what usage calls may look like.
> They are documentation examples, not guaranteed runnable snippets.

1) Define a model

```
using CoreRelm.Attributes;

[RelmTable("people")]
public class PersonModel : RelmModel
{
    [RelmColumn]
    public string FirstName { get; set; } = string.Empty; // automatic column
name: "first_name"

    [RelmColumn]
```

```
    public string LastName { get; set; } = string.Empty; // automatic column
name: "last_name"

    [RelmColumn("user_level")]
    public int Level { get; set; } = 0; // overridden column name: "user_level"
}
```

The `Id`, `Active`, `InternalId`, `CreateDate`, and `LastUpdated` fields are automatically defined in the base `RelmModel` class. Inherit from the `RelmModelClean` base class if you want to opt out of these default fields.

2) Define a context

```
using CoreRelm;
using CoreRelm.Models;

public class ExampleContext(RelmContextOptions options) : RelmContext(options)
{
    public RelmDataSet<PersonModel>? People { get; set; }
}
```

3) Build options and create a context

```
using var context = new RelmContextOptionsBuilder("localhost", "example_database",
"root", "password")
    .Build<ExampleContext>();
```

This context will eagerly initialize every data set (but not retrieve data) upon build as `AutoInitializeDataSets` is enabled by default. You can disable this behavior in the options builder if you prefer to initialize data sets manually.

You can initialize RelmContextOptionsBuilder with the following parameter types:

- `string server`, `string database`, `string userId`, `string password`
- `string server`, `string port`, `string database`, `string userId`, `string password`
- `string connectionString`
- `string configConnectioName`
- `enum connectionStringType`
- `MySqlConnection connection`
- `MySqlConnection connection, MySqlTransaction transaction`
- `RelmContextOptions options`
- `IRelmContext relmContext`

4) Read and write data

```
var person = context.People.New();
person.FirstName = "Ada";
person.LastName = "Lovelace";
person.WriteToDatabase(context);

var found = context.People.Find(person.InternalId);

var results = context.People
    .Where(x => x.LastName == "Lovelace")
    .OrderBy(x => x.FirstName)
    .Load();
```

5) Use an explicit transaction

```
using CoreRelm.Options;

using var context = new RelmContextOptionsBuilder()
    .SetServer("localhost")
    .SetDatabase("ExampleDatabase")
    .SetUserId("root")
    .SetPassword("password")
    .AutoInitializeDataSets(false) // turn off to lazy-initialize data sets, enabled
by default
    .Build<ExampleContext>();

context.BeginTransaction();

try
{
    var person = context.GetDataSet<PersonModel>().New();
    person.FirstName = "Grace";
    person.LastName = "Hopper";
    person.Save();

    context.CommitTransaction();
}
catch
{
    context.RollbackTransaction();

    // Optional diagnostics (where applicable):
    if (RelmHelper.HasError)
    {
```

```
        var lastErrorMessage = RelmHelper.LastExecutionError;
        var lastException = RelmHelper.LastExecutionException;

        // log or inspect errors/excpetions as needed
    }

    throw;
}
```

In this example, we set `AutoInitializeDataSets` to `false` to lazy-initialize data sets. `GetDataSet` is used to initialize the lazy-initialized `People` data set manually. Remove the `AutoInitializeDataSets` option or set it to `true` (the default) to automatically initialize data sets on context options build.

Features

CoreRelm includes support for:

- Attribute-based model mapping
- Context and dataset patterns
- Identity and key handling
- Foreign key relationships and reference loading
- DTO mapping patterns
- Data loader patterns
- Bulk write helpers
- Context configuration through options builders
- Helper APIs for metadata and mapping support
- Migrations tooling and model/schema workflows
- MySQL-focused provider behavior for MySQL 8+

What the Quickstart demonstrates

The `examples/CoreRelm.Quickstart` project is intended to show what API usage calls may look like.

It is a documentation-style example project and is not required to compile or run.

The Quickstart is used to demonstrate:

- Context usage patterns
- Connection examples
- Dataset operations
- Data retrieval call shapes
- DTO usage patterns
- Data loader usage patterns
- Foreign key loading patterns

- Identity examples
- Bulk writer examples
- Model reset examples
- Model property naming examples
- Attribute usage examples
- Migrations examples (planned and included as documentation-style examples)

Feature Matrix

This matrix shows where to look for each type of information.

Runtime ORM usage

- **README**
  - Introductory examples
  - Core concepts
  - Basic usage patterns
- **Quickstart**
  - Documentation-style API call examples
  - Broader examples across public features
- **API docs**
  - Type/member reference details
- **Tests**
  - Behavioral coverage and edge cases

Migrations

- **README**
  - High-level migrations overview and example usage shape
- **Quickstart**
  - Documentation-style migrations examples
- **API docs**
  - Tooling and type/member reference details
- **Tests**
  - Parsing/planning/tooling behavior validation

Error handling and diagnostics

- **README**
  - Recommended handling patterns
- **Quickstart**
  - Example call shapes
- **API docs**

    - Member reference
- **Tests**
    - Behavior verification

What is documented here

This README is focused on:

- What CoreRelm is
- When to use it
- A fast “first look” setup
- High-level feature overview
- Transaction and error handling guidance
- Migrations overview
- Pointers to deeper documentation

For detailed API reference, use the generated docs in `docs/api`.

For broader usage examples, use `examples/CoreRelm.Quickstart`.

For behavior and edge cases, review `CoreRelm.Tests`.

Quickstart examples

Quickstart examples are intentionally documentation-first.

They are meant to show:

- What a call may look like
- How a feature is typically used
- The shape of the API for a common scenario

They are not required to be executable examples.

This allows the Quickstart project to demonstrate public features clearly without forcing every sample to be fully wired to a live database.

Migrations

CoreRelm includes migrations-related APIs and tooling as part of the public surface.

Migrations are documented in three places:

- This README (high-level overview)
- Quickstart examples (documentation-style usage examples)

- API docs (`docs/api`) for full type/member reference

Migrations usage shape (example)

> This example demonstrates intent and call shape only.

```
// Example only: demonstrate what migrations tooling usage may look like.

var migrationTool = new RelmMigrationTooling();
var result = migrationTool.GenerateMigration(/* model set / options */);

if (!result.Success)
{
    // inspect errors
}

var applyResult = migrationTool.ApplyMigrations(/* connection / options */);
```

Migrations guidance

- Keep migrations explicit and reviewable
- Treat generated SQL and migration steps as part of your deployment artifacts
- Use tests and staging verification before production execution
- Prefer clear naming and consistent migration organization

Error handling

CoreRelm is designed to work well with explicit error handling and transaction boundaries.

Recommended pattern

- Start a transaction explicitly when the operation spans multiple writes
- Commit only after all operations succeed
- Roll back on failure
- Inspect context/tooling error state where applicable
- Re-throw or wrap exceptions at application boundaries as appropriate

Example pattern

```
using var context = new ExampleContext(options);
context.BeginTransaction();

try
```

```
{
    // multiple operations
    // context.People.Add(...)
    // context.SaveChanges() or model.Save()

    context.CommitTransaction();
}
catch (Exception ex)
{
    context.RollbackTransaction();

    // Optional: inspect CoreRelm error state where applicable
    // var hasError = context.HasError;
    // var lastError = context.LastExecutionError;
    // var lastException = context.LastExecutionException;

    throw;
}
```

Error handling notes

- Prefer explicit transactions for multi-step write workflows
- Do not assume partial writes are safe without a transaction
- Log both application exceptions and ORM/database diagnostic details when available
- Keep error-handling paths simple and consistent across contexts/services

CoreRelm vs EF and Dapper

CoreRelm is not intended to replace every ORM pattern.

It is best viewed as a different tradeoff.

Compared to Entity Framework

CoreRelm generally favors:

- More explicit behavior
- Less hidden query translation complexity
- A simpler mental model for developers who want direct control
- Fewer framework conventions driving runtime behavior

Entity Framework generally offers:

- Rich ecosystem and tooling
- Deep change tracking and LINQ support

- Broad provider support
- Higher-level abstraction for large app architectures

Compared to Dapper

CoreRelm generally offers:

- More built-in structure around models and datasets
- Attribute-based mapping and metadata
- A more consistent ORM-style workflow for repeated patterns
- A unified place for schema-related metadata and runtime usage patterns

Dapper generally offers:

- Extremely lightweight micro-ORM usage
- Very direct SQL-first mapping
- Minimal abstraction overhead
- Great fit for teams that want to hand-author most SQL

When CoreRelm is a strong fit

- You want attribute-based models and consistent mapping
- You want ORM helpers without giving up explicit control
- You prefer readable, predictable data-access patterns
- You are targeting MySQL 8+ and want a focused provider story

Contributing

Contributions are welcome.

General guidelines

- Keep changes focused and well-scoped
- Prefer clear, explicit code over clever abstractions
- Add or update tests for behavior changes
- Update Quickstart examples for public API changes
- Update README and API docs references when relevant

Documentation contributions

If you add or change a public feature, please update:

- `README.md` for high-level guidance
- `examples/CoreRelm.Quickstart` for usage-shape examples
- `docs/api` generated documentation (as applicable)

Pull request suggestions

- Include a short summary of what changed
- Note any public API changes
- Note any migration-related implications
- Include before/after usage examples when possible

Roadmap

The roadmap will evolve as the library grows.

Current areas of focus include:

- Expanding Quickstart coverage for all public features
- Migrations examples and documentation depth
- Additional documentation consistency across README, Quickstart, and API docs
- Continued runtime and tooling refinement
- API polish and developer experience improvements

Related project

If you are looking for the .NET Framework 4.8 version, see **SimpleRelm**.

CoreRelm is the Core 8+ line and is the active path for this repository.

Introduction

CoreRelm is a lightweight, attribute-based ORM for C# that stays close to ADO.NET style development while reducing repetitive data access code.

It is designed for developers who want:

- Strong control over SQL and transactions
- Attribute-driven model mapping
- Predictable behavior without heavy abstraction
- A simple context and dataset workflow
- A MySQL-focused ORM for .NET 8+

CoreRelm favors explicitness over magic. The library gives you structure for models, datasets, options, helpers, and migrations, while still keeping the database visible in the way you work.

What CoreRelm is trying to be

CoreRelm is intended to sit in the space between writing raw ADO.NET everywhere and adopting a heavier ORM abstraction.

At a high level, it provides:

- Attribute-based model metadata
- A `RelmContext`-centered workflow
- `RelmDataSet` access patterns for models
- Options-based context configuration
- Utility helpers for common relational tasks
- Migrations tooling and schema-oriented APIs
- Support for both eager and lazy context initialization patterns

The goal is not to hide the database. The goal is to make common relational work more structured, more readable, and more repeatable.

Who CoreRelm is for

CoreRelm is a strong fit when you want:

- A practical ORM that still feels close to the database
- Clear transaction boundaries
- Attribute-driven mapping instead of large external configuration systems
- A model and dataset workflow that is simple to reason about
- A codebase where explicit behavior is preferred over implicit conventions

It may be especially appealing if you like the directness of lower-level database code but want more consistency and reuse around model definition, querying patterns, relationships, and schema workflows.

Supported platform and provider model

CoreRelm currently targets:

- .NET 8+
- MySQL 8+ only

The current provider story is intentionally focused. The library and documentation should be read with MySQL as the active provider target.

Core building blocks

Most of the library revolves around a few core ideas:

`RelmContext`

`RelmContext` is the main runtime entry point. It represents a configured database context and is the place where connection behavior, transaction behavior, dataset access, and lower-level execution helpers come together.

`RelmDataSet`

A `RelmDataSet` represents a model-oriented access surface for a particular entity type. This is where many common operations such as querying, loading, creating, and referencing related data are expressed.

`RelmModel`

A `RelmModel` represents a mapped entity. Attributes applied to the model and its members describe how the model relates to the database.

`RelmContextOptions` and `RelmContextOptionsBuilder`

Context options define how a CoreRelm context behaves. The options builder is the main way to configure connection details, auto-open behavior, initialization behavior, and related settings.

Design style

CoreRelm follows a few broad design preferences:

- Explicit transaction handling is preferred
- Configuration should be readable and close to usage
- Metadata should live near the model when practical
- Examples should show the shape of the API clearly

- Runtime behavior should be understandable without guessing

That design style is visible in the QuickStart project as well. The examples there are documentation-style examples meant to show what a call may look like. They are not intended to serve as a polished end-to-end sample application.

What you will find in this documentation

This guide is organized to help different kinds of readers move through the library effectively.

If you are new to CoreRelm:

1. Start with Getting Started
2. Review the Core Concepts section
3. Move into the Working with Data section

If you already know what you need:

- Use the guide pages for narrative explanations and recommended patterns
- Use the API reference for type and member details
- Use `examples/CoreRelm.QuickStart` to see documentation-style usage examples

CoreRelm versus EF and Dapper

CoreRelm is not trying to replace every ORM style.

Compared to Entity Framework, CoreRelm generally favors:

- More explicit behavior
- Less hidden runtime convention
- A simpler mental model for developers who want direct control
- A closer relationship to relational and connection-level details

Compared to Dapper, CoreRelm generally offers:

- More built-in structure around models and datasets
- Attribute-based mapping and metadata
- A more unified ORM-style surface for repeated patterns
- Schema- and migrations-related concepts in the same ecosystem

In practice, CoreRelm is best understood as a deliberate tradeoff: more structure than a very thin micro-ORM, but more explicitness than a heavier convention-driven ORM.

Where to go next

The best next step is Getting Started.

That page will walk through:

- how CoreRelm is configured
- how contexts are initialized
- what a minimal usage flow looks like
- where the QuickStart project fits into the documentation

Namespace CoreRelm

Classes

RelmHelper

Provides a collection of helper methods and properties for interacting with relational databases.